# Yenepoya Institute of Arts, Science, Commerce, and Management

## Project – High Level Design

## on

## Dark Web Monitoring for Agricultural Assets

**Student Name:** Mr. Alan Shibu

**Industry Mentor:** Mr. Shashank

**Guided by:** Ms. Divya N Shetty

# Table of Contents


| No. | Section Title | Page |
|---|---|---|
| **1** | **Introduction** | 3 |
| 1.1 | Scope of the Document | 3 |
| 1.2 | Intended Audience | 4 |
| 1.3 | System Overview | 4 |
| **2** | **System Design** | 5 |
| 2.1 | Application Design | 5 |
| 2.2 | Process Flow | 6 |
| 2.3 | Information Flow | 9 |
| 2.4 | Components Design | 10 |
| 2.5 | Key Design Considerations | 12 |
| 2.6 | API Catalogue | 13 |
| **3** | **Data Design** | 13 |
| 3.1 | Data Model | 13 |
| 3.2 | Data Access Mechanism | 16 |
| 3..3 | Data Retention Policies | 17 |
| 3.4 | Data Migration | 17 |
| **4** | **Interfaces** | 18 |
| **5** | **State and Session Management** | 20 |
| **6** | **Caching** | 22 |
| **7** | **Non-Functional Requirements** | 24 |
| 7.1 | Security Aspects | 24 |
| 7.2 | Performance Aspects | 26 |
| **8** | **References** | 28 |

# 1. Introduction

The increasing digitization of agricultural systems has led to the storage and exchange of sensitive data, making them vulnerable to cybersecurity threats such as data leaks and unauthorized access. The dark web serves as a platform where such compromised data is often traded, posing significant risks to agricultural organizations.
This project Dark Web Monitoring for Agricultural Assets aims to develop a proactive system that monitors dark web sources to detect potential data leaks. The system leverages secure crawling techniques, web scraping, and Natural Language Processing (NLP) to identify relevant threats and generate timely alerts.

## 1.1 Scope of the Document

This document presents the High-Level Design (HLD) of the Dark Web Monitoring for Agricultural Assets system. It defines the overall system architecture, major components, and data flow involved in monitoring and detecting potential data leaks related to agricultural entities on the dark web.
In Scope:

- System architecture and module design
- Dark web data collection approach
- Data processing and filtering mechanisms
- Alert generation and reporting workflow
- Security and performance considerations

Out of Scope:

- Low-level implementation details and source code
- Advanced threat mitigation or automated response systems
- Full-scale dark web coverage or real-time global monitoring
- Legal enforcement or investigative actions

The document focuses on providing a conceptual and structural understanding of the system while ensuring ethical and controlled access to dark web resources.

## 1.2 Intended Audience

This document is intended for the following stakeholders:

- Cybersecurity Analysts – to understand the threat detection, monitoring workflow, and potential risks identified from dark web sources
- Developers – to implement and integrate various system components based on the defined architecture
- System Architects – to review, validate, and enhance the overall system design and scalability
- Project Evaluators / Mentors – to assess the feasibility, design quality, and effectiveness of the proposed solution

## 1.3 System Overview

The Dark Web Monitoring System for Agricultural Assets is designed to proactively detect potential data leaks by monitoring hidden sources such as onion sites and underground forums. The system focuses on identifying sensitive information related to agricultural organizations and assets to mitigate cybersecurity risks.

The system operates through a structured pipeline. Initially, a Tor-enabled crawler securely accesses dark web sources and extracts relevant data. The collected data is then passed through a processing module that performs data preprocessing, keyword-based filtering, and Natural Language Processing (NLP) analysis to identify potential threats and sensitive information.

Once relevant matches are detected, the system stores the processed data and triggers real-time alerts through communication channels such as email and Slack. Additionally, a dashboard interface provides visualization of detected threats, patterns, and system insights, enabling stakeholders to monitor risks effectively.

Overall, the system facilitates early detection of potential security breaches, allowing organizations to take timely preventive actions and strengthen their cybersecurity posture.

# 2. System Design

This section presents the overall design of the Dark Web Monitoring System, describing its architecture, components, and data flow. It provides a high-level understanding of how different modules interact to collect, process, and analyze data from dark web sources. The section also highlights key design considerations and system functionalities required to ensure efficient and secure operation.

## 2.1 Application Design

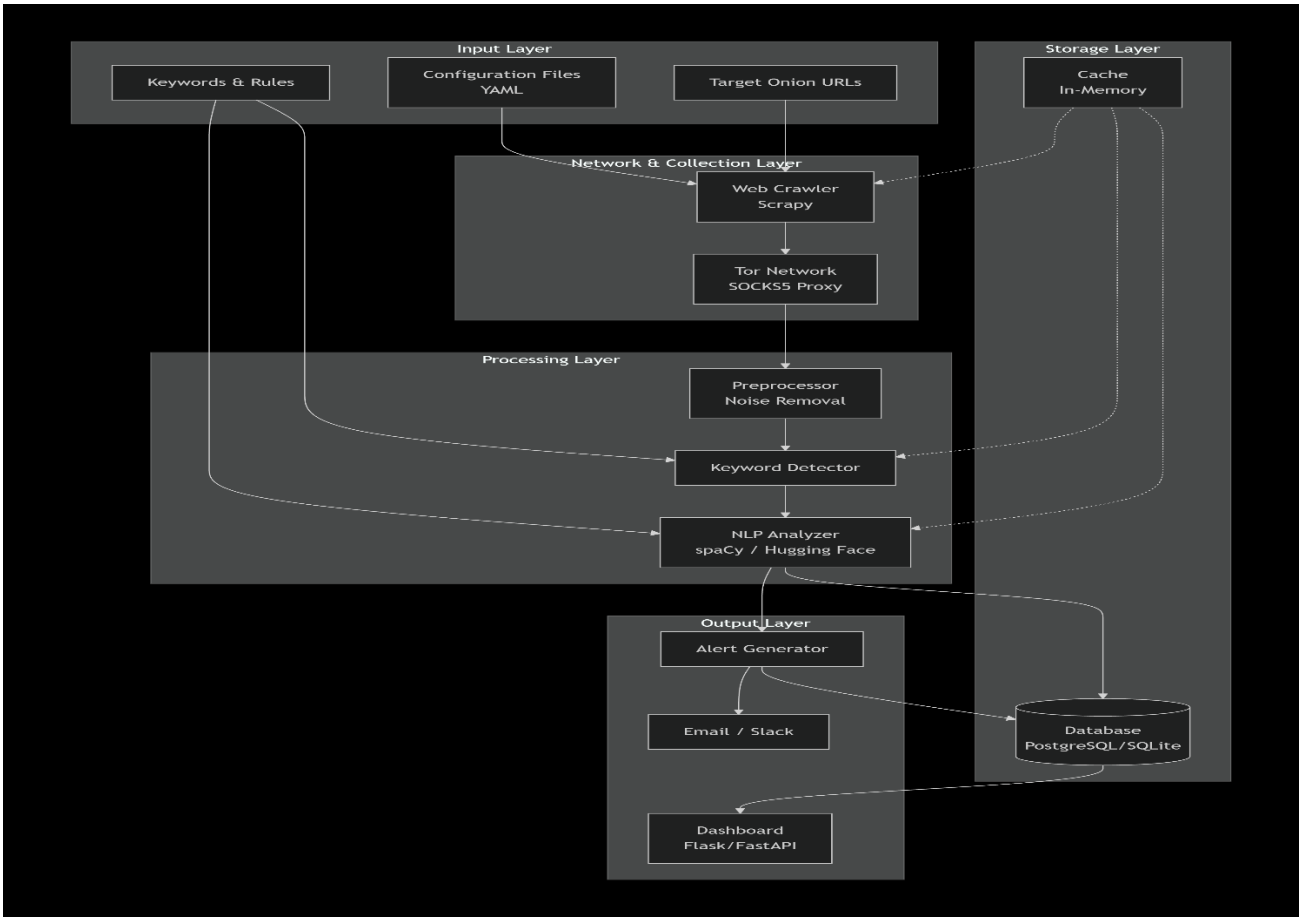
The Dark Web Monitoring System is designed using a modular and pipeline-based architecture, where each component is responsible for a specific stage in the data processing lifecycle. This design ensures scalability, flexibility, and ease of maintenance by enabling independent development and deployment of system modules.

The system is composed of multiple interconnected modules that collaboratively perform data collection, processing, storage, and alerting. The workflow begins with the configuration module, which initializes system parameters such as target onion URLs, keywords, and alert preferences.

A Tor-enabled network layer establishes a secure and anonymous connection to access dark web sources. The web crawling module, built using Scrapy, retrieves relevant data from these sources. The collected data is then passed to the processing module, where preprocessing techniques remove noise and irrelevant content, followed by keyword-based filtering and Natural Language Processing (NLP) analysis to identify sensitive information related to agricultural assets.

The processed data is stored in a database for persistence and further analysis. Upon detecting relevant threats or data leaks, the alert generation module triggers notifications through channels such as email and Slack. In parallel, a dashboard module provides visualization of alerts, trends, and system insights, enabling users to monitor and respond effectively.

This modular architecture promotes separation of concerns and allows the system to scale efficiently. It also supports future enhancements such as integrating additional data sources, improving detection mechanisms, and incorporating advanced analytics capabilities.




## 2.2 Process Flow

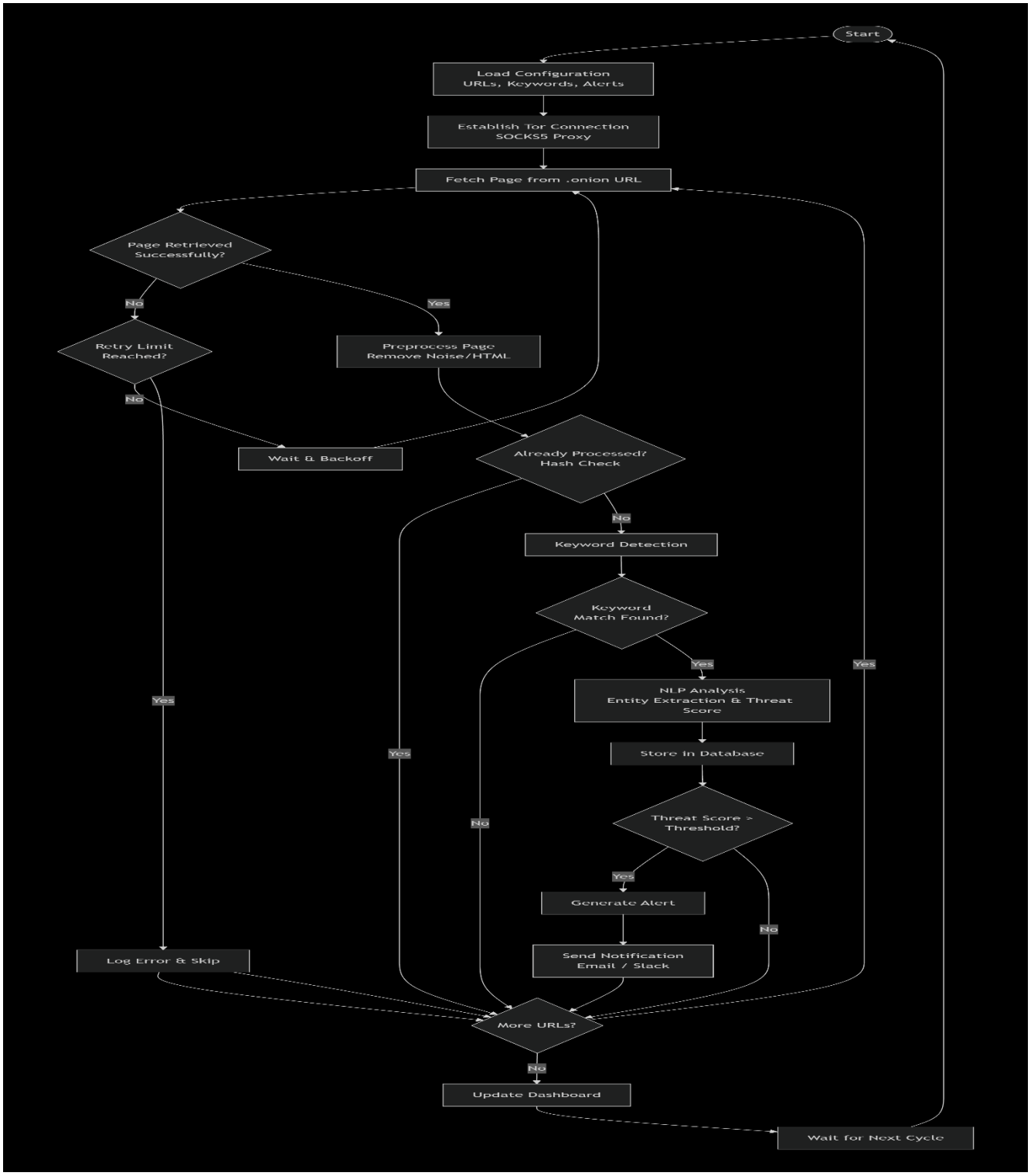
The process flow of the Dark Web Monitoring System describes the sequence of operations performed to detect potential data leaks from dark web sources.

The system follows a structured and continuous pipeline approach, where data is collected, processed, analyzed, and acted upon.

Initially, the system loads configuration settings such as target onion URLs, keywords, and alert parameters. A secure connection is then established through the Tor network to enable anonymous access to dark web sources. The web crawler retrieves data from the specified sources and passes it to the preprocessing stage, where noise and irrelevant content are removed.

The cleaned data is subsequently analyzed using keyword-based filtering and Natural Language Processing (NLP) techniques to identify sensitive information related to agricultural assets. If a relevant match is detected, the data is stored in the database and an alert is generated through configured communication channels. If no match is found, the data is discarded to optimize storage and processing efficiency.

Finally, detected incidents and alerts are visualized on the dashboard, allowing users to monitor threats and analyze trends. This process runs continuously or at scheduled intervals to ensure timely detection of potential security breaches.


Start
Load Configuration
URLs, Keywords, Alerts
Establish Tor Connection
SOCKS5 Proxy
Fetch Page from .onion URL
Page Retrieved
Successfully?
No
Yes
Retry Limit
Reached?
Preprocess Page
Remove Noise/HTML
No
Wait & Backoff
Already Processed?
Hash Check
No
Keyword Detection
Keyword
Match Found?
Yes
Yes
NLP Analysis
Entity Extraction & Threat
Score
Yes
Store in Database
No
Threat Score >
Threshold?
Yes
Generate Alert
No
Send Notification
Email / Slack
Log Error & Skip
More URLs?
No
Update Dashboard
Wait for Next Cycle

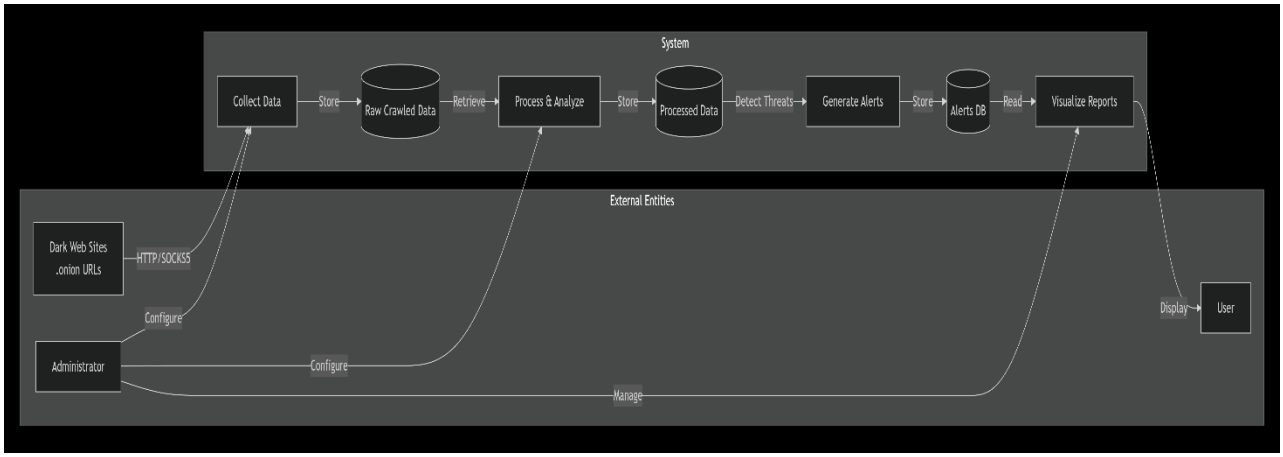
## 2.3 Information Flow

The information flow of the system describes how data is collected, transformed, and converted into actionable insights throughout the monitoring pipeline.

Input:
The system receives input in the form of target onion URLs, predefined keywords related to agricultural assets, and configuration settings such as alert parameters. Additionally, raw textual data is gathered from dark web sources during the crawling process.

Processing:
The raw data collected from dark web sources is first passed through the preprocessing stage, where noise, irrelevant content, and unwanted elements are removed to produce cleaned data. This cleaned data is then analyzed using keyword detection and Natural Language Processing (NLP) techniques to extract meaningful information, identify sensitive content, and detect relevant entities associated with agricultural assets. The data is thus transformed from unstructured raw text into structured and analyzable information.

Output:
If relevant matches are identified, the processed data is stored in the database for persistence and further analysis. Alerts are generated and communicated through channels such as email or Slack. The final output is also presented on a dashboard in the form of visual insights, enabling effective monitoring and decision-making.



## 2.4 Components Design

The system is divided into multiple modules, each responsible for a specific functionality within the overall pipeline. This modular design ensures flexibility, scalability, and ease of maintenance by enabling independent development and updates of components.

1. Configuration Module
   Responsible for loading system configurations such as target onion URLs, keywords, and alert settings from configuration files.

2. Tor Network Module
   Establishes a secure and anonymous connection using the Tor network (via SOCKS5 proxy) to enable safe access to dark web sources.

3. Web Crawling Module
   Implements a crawler (Scrapy-based) to fetch data from onion sites and underground forums through the Tor network.

4. Data Preprocessing Module
   Cleans and normalizes extracted data by removing noise, HTML tags, and irrelevant content to prepare it for analysis.

5. Keyword Detection Module
   Identifies predefined keywords such as organization names, domains, and sensitive terms within the processed data.

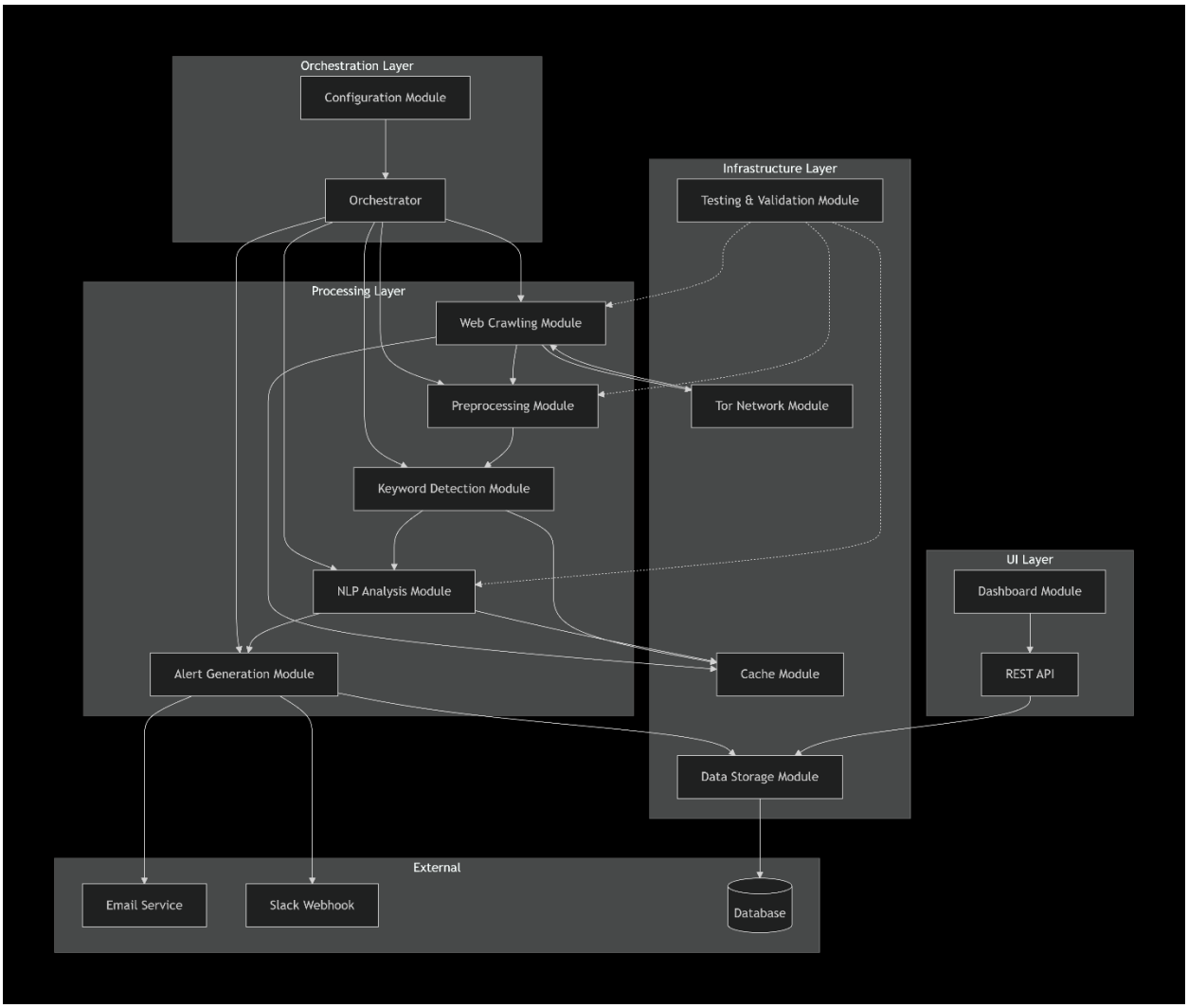
6. NLP Analysis Module
   Applies Natural Language Processing techniques to extract meaningful insights, identify entities, and detect contextual relevance in the data.

7. Data Storage Module
   Stores processed data, detected incidents, and metadata in a database for persistence and further analysis.

8. Alert Generation Module
   Triggers notifications via communication channels such as emails or Slack when potential threats are detected.

9. Dashboard Module
   Provides a user interface to visualize alerts, trends, and insights for monitoring and decision-making.

10. Testing and Validation Module
    Ensures system reliability through testing and validation of data collection, processing, and alert mechanisms.



## 2.5 Key Design Considerations

The design of the Dark Web Monitoring System is guided by several key factors to ensure secure, efficient, and reliable operation.

1. Security
   The system leverages the Tor network to ensure anonymous access to dark web sources and protect the identity of the host system. Sensitive information such as credentials and API keys is securely managed using environment variables, and appropriate access controls are implemented to prevent unauthorized data access.

2. Scalability
   The modular and loosely coupled architecture enables horizontal scaling by adding additional crawlers, data sources, or processing modules without affecting existing components. This allows the system to handle increasing data volumes and evolving monitoring requirements.

3. Performance
   Efficient crawling mechanisms and optimized data preprocessing techniques ensure timely data collection and analysis. The system is designed to minimize latency and handle large volumes of data efficiently.

4. Reliability
   The system incorporates error handling, retry mechanisms, and fault-tolerant design to manage network failures and inaccessible dark web sources. Logging and monitoring are used to track system health and detect failures.

5. Maintainability
   The modular design promotes separation of concerns, making it easier to update, debug, or replace individual components without impacting the entire system. This improves long-term maintainability and development efficiency.

6. Ethical and Legal Considerations

   The system is designed to operate within ethical and legal boundaries by focusing only on passive monitoring of publicly accessible dark web content. No intrusive or illegal activities are performed, ensuring compliance with cybersecurity and data protection standards.

## 2.6 API Catalogue

The Dark Web Monitoring System exposes a set of RESTful APIs to enable interaction between system components and external interfaces such as dashboards. These APIs support key functionalities including data retrieval, alert management, and system monitoring. The following table provides a high-level overview of the available API endpoints.

| Endpoint | Method | Description |
|---|---|---|
| /api/statistics | GET | Retrieve system-wide monitoring statistics |
| /api/alerts | GET | List security alerts with filtering |
| /api/alerts/{id} | PUT | Update alert status |
| /api/search | GET | Search across alerts and data |
| /api/recent-pages | GET | Retrieve recently crawled pages |
| /api/charts/threat-trend | GET | Provide threat trend data for visualization |

# 3. Data Design

This section describes how data is structured, stored, and managed within the Dark Web Monitoring System. It outlines the data model, storage mechanisms, and data handling practices used to support efficient processing, retrieval, and analysis of detected information.

## 3.1 Data Model

The system uses a structured data model to store and manage information collected from dark web sources. Unstructured data obtained from crawling is processed and transformed into structured records for efficient analysis and storage.

The key entities in the system include Crawled Pages, Processed Content, Keyword Matches, NLP Results, and Alerts.

Key Entities

1. Crawled Page
    - Id
    - url
    - crawled_at
2. Processed Content
    - Id
    - Page_id
    - Cleaned_text
3. Keyword Matches
    - Id
    - Page_id
    - Keyword
    - Category
4. NLP Results
    - Id
    - Page_id
    - threat_score
    - threat_level
5. Alerts
    - Id
    - Page_id
    - Alert_level
    - Risk_score
    - Generated_at

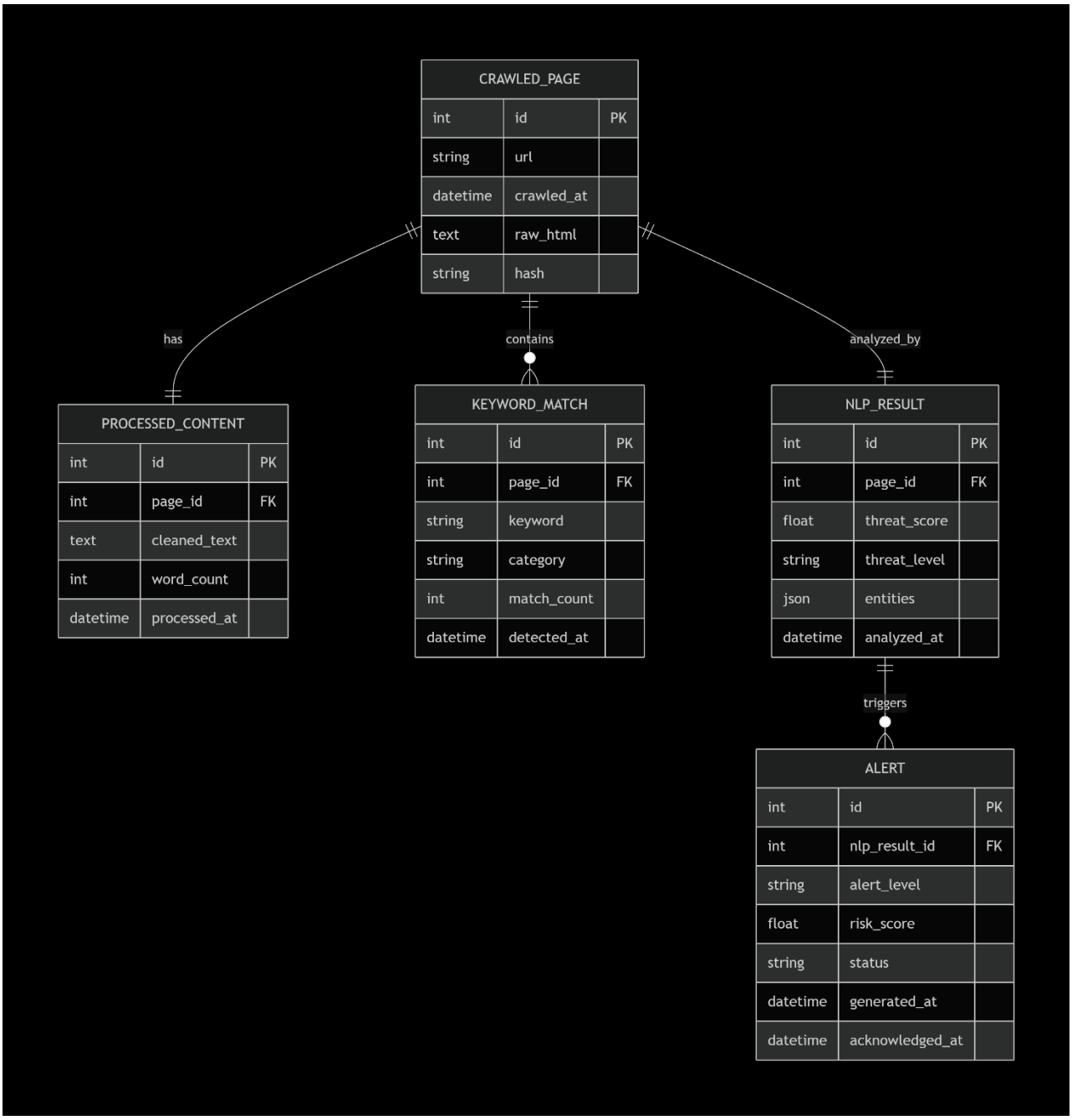
Relationships

Crawled → Processed Content

Crawled → Keyword Matches

Crawled → NLP Results

NLP Results → Alerts


CRAWLED_PAGE
int id PK
string url
datetime crawled_at
text raw_html
string hash
has
contains
analyzed_by
PROCESSED_CONTENT
int id PK
int page_id FK
text cleaned_text
int word_count
datetime processed_at
KEYWORD_MATCH
int id PK
int page_id FK
string keyword
string category
int match_count
datetime detected_at
NLP_RESULT
int id PK
int page_id FK
float threat_score
string threat_level
json entities
datetime analyzed_at
triggers
ALERT
int id PK
int nlp_result_id FK
string alert_level
float risk_score
string status
datetime generated_at
datetime acknowledged_at

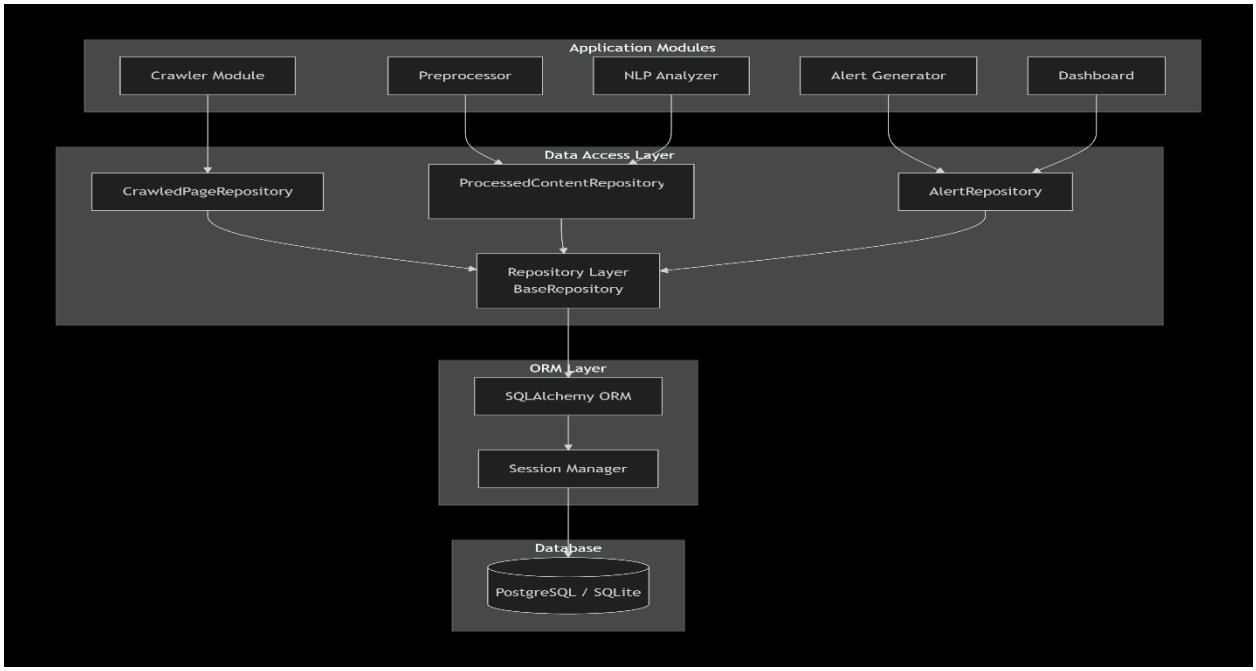
## 3.2 Data Access Mechanism

The system uses an Object-Relational Mapping (ORM) approach for interacting with the database, providing an abstraction layer between application modules and the underlying data storage. This ensures modularity, maintainability, and database independence.

SQLAlchemy ORM is used to handle database operations such as data insertion, retrieval, and updates. The system supports multiple database environments, including SQLite for development and PostgreSQL for production.

A repository-based data access layer is implemented to centralize all database interactions. Application modules such as the crawler, preprocessing unit, NLP analyzer, and alert generator interact with the database through this layer, ensuring separation of concerns and clean architecture.

The data access mechanism also incorporates connection management and transaction handling to ensure reliable and efficient database operations.



## 3.3 Data Retention Policies

The system defines retention policies for each data category to balance storage efficiency, historical analysis, and audit requirements. Frequently changing operational data is retained for a shorter period, while alerts and audit records are preserved longer for investigation and compliance purposes.

| Data Type | Retention Period | Action |
|---|---|---|
| Crawled Pages | 90 days | Delete |
| Processed Content | 90 days | Delete |
| Keyword Matches | 180 days | Archive |
| NLP Results | 180 days | Retain |
| Alerts | 365 days | Retain |
| Scan History | 365 days | Retain |

The system automatically removes or archives expired records through scheduled cleanup processes. Historical security alerts and scan logs are preserved to support incident investigation and audit requirements. Archived records are moved to long-term storage instead of remaining in the primary database.

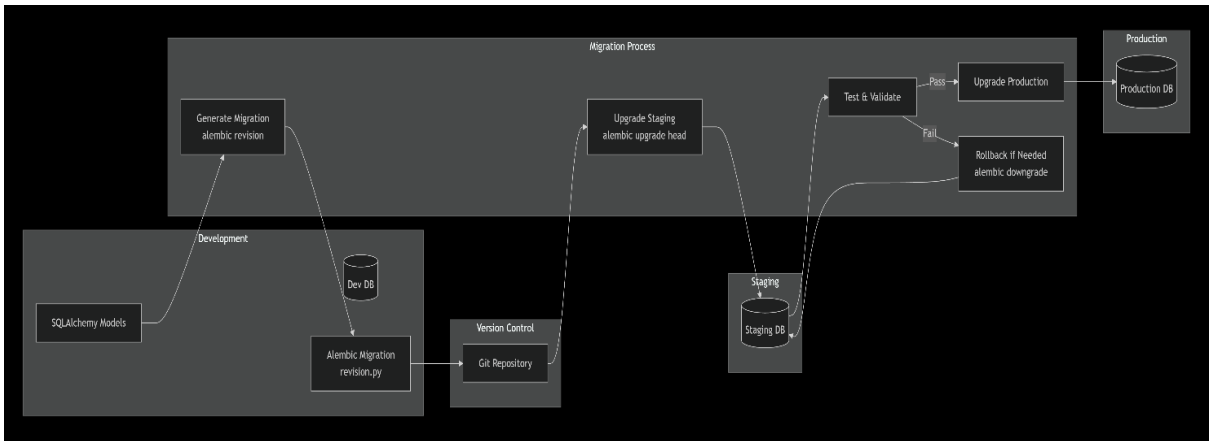
## 3.4 Data Migration

The system supports data and schema migration to accommodate evolving requirements and ensure compatibility across different environments. Migration processes handle structural changes to the database as well as transformation or movement of existing data.

Alembic is used as the migration tool, integrated with the SQLAlchemy ORM. It enables version-controlled schema updates and ensures consistency between development, staging, and production environments.

The system supports multiple types of migration, including schema migration (e.g., adding or modifying tables), data migration (e.g., transforming existing records), and environment migration (e.g., moving data between development and production systems).

All migrations follow a controlled workflow where changes are generated, reviewed, and applied sequentially to maintain database integrity and avoid inconsistencies.




## 4. Interfaces

The system interfaces are categorized into four types:

- External Interfaces – Interaction with external systems (Tor, email, webhooks)
- Internal Interfaces – Communication between system modules
- User Interface – Dashboard for monitoring and analysis
- Configuration Interface – YAML-based configuration management

❖ External Interfaces

These interface enable communication with systems outside the application.

| | |
|---|---|
| Tor Network Interface | • Provides anonymous communication using Tor<br>• Uses SOCKS5h protocol via local Tor daemon<br>• Support identity rotation for privacy |
| Dark Web Site Interface | • Fetch content from .onion websites<br>• Uses HTTP/HTTPS requests<br>• Includes retry and delay mechanisms |

| | |
|---|---|
| Email Interface | • Send alert notifications to users<br>• Uses SMTP with secure connection<br>• Support multiple recipients |
| Webhook Interface | • Sends alerts to platforms like Slack<br>• Uses HTTP POST with JSON payload<br>• Enable integration with external systems |

- Internal Interfaces

  Internal interfaces define communication between system modules

| **Caller** | **Callee** | **Purpose** |
|---|---|---|
| Main | Crawler | Start crawling process |
| Crawler | Preprocessor | Clean extracted content |
| Preprocessor | Keyword Detector | Detect sensitive keywords |
| Keyword Detector | NLP Analyzer | Analyze context and intent |
| NLP Analyzer | Alert Generator | Generate alerts |
| Alert Generator | Storage | Save alerts |
| Dashboard | Storage | Fetch alerts and statistics |

- Data Transfer Objects

  Data Transfer Objects are used to transfer structured data between modules

| **Data Transfer Objects** | **Purpose** |
|---|---|
| CrawledPage | Stores raw page data |
| ProcessedDocument | Stores cleaned content |
| DocumentMatchResult | Stores keyword detection results |
| NLPResult | Stores analysis output |
| Alert | Stores alert details |

- User Interface

  Dashboard

  The system provides a web-based dashboard for monitoring

  Features:

  → View alerts and threat levels
  → Search and filter data
  → View statistics and trends

→ Acknowledge alerts

API Endpoints

| Endpoint | Purpose |
|---|---|
| /api/statistics | Get system metrics |
| /api/alerts | Fetch alerts |
| /api/charts | Get visualization data |
| /api/recent-pages | View recent crawled pages |

❖ Configuration Interface

The system uses YAML files for configuration

Key Configuration Files

| File | Purpose |
|---|---|
| Settings.yaml | General system settings |
| Keywords.yaml | Keywords to monitor |
| tor_config.yaml | Tor network settings |
| alerts.yaml | Alert rules and threshold |

## 5. State and Session Management

The system manages state across multiple layers to ensure efficient processing and persistence.

| Layer | State Type | Storage | Scope |
|---|---|---|---|
| Crawler | Crawl progress | In-memory | Per crawl session |
| Tor | Network identity | In-memory | Application runtime |
| Database | Pages, alerts, entities | Persistent | Permanent |
| Dashboard | User Session | In-memory | Client Session |
| Alert | Alert lifecycle | Database | Permanent |

- Crawler State Management

  The crawler maintains runtime state for efficient crawling.

  Key States:
  Visited URLs (to avoid duplicates)
  URL Queue (pending pages)
  Domain visit counts

Characteristics:
Stored in memory
Lost on restart
Ensure efficient crawling

- Database Session Management

  The system uses a session-based database access pattern.

  Features:
  Automatic commit on success
  Rollback on failure
  Connection pooling for performance

- Alert State Management

  Key Fields

  Status (new, escalated, resolved)
  Acknowledged (true/false)
  Escalation_level

- Dashboard Session Management

  The dashboard maintains user session state.

  Stored Data:
  Session ID

  Filter preferences
  Active connections (WebScoket)

  Characteristics:
  Stored in memory
  Lost on disconnect
  Auto-reconnect supported

## 6. Caching

Caching is used to temporarily store frequently accessed or computationally expensive data so that repeated processing can be avoided.

In the Dark Web Agriculture Monitor, caching helps to:

- Improve system performance by reducing redundant operations
- Minimize repeated crawling of identical content
- Reduce load on external systems such as the Tor network and NLP models
- Optimize database and network resource usage

Types of Caching Used

| **Cache Type** | **Purpose** | **Storage** | **TTL** |
|---|---|---|---|
| Content Deduplication Cache | Avoid processing Duplicate pages | In-memory | Session |
| Alert Deduplication Cache | Prevent duplicate alerts | In-memory | 60 minutes |
| NLP Model Cache | Reuse loaded NLP models | In-memory | Permanent |
| Regex Pattern Cache | Reuse compiled patterns | In-memory | Permanent |
| Data Connection Pool | Reuse DB connections | Memory | 1 hours |
| Session Cache | Reuse HTTP connections | Memory | Session |

Key Caching Mechanisms

1. Content Deduplication Cache

    - Stores hash values of recently crawled content
    - Prevent reprocessing identical or similar pages
    - Improves crawling efficiency

2. Alert Deduplication Cache

    - Stores recent alerts using a unique hash
    - Prevents duplicate alerts within a defined time window
    - Reduces alerts noise for analysts

3. NLP Model Cache

    - NLP models (spcaCy, Hugging Face) are loaded once and resused
    - Eliminates repeated model for loading time
    - Significantly improves processing speed

4. Regex Pattern Cache

    - Keyword patterns are compiled once at startup
    - Reused for all document processing
    - Reduces repeated computation overhead

5. Database Connections Pool Cache

    - Maintain a pool of reusable database connections
    - Avoid overhead of creating new connections for each request
    - Improves query performance

6. Session Cache (HTTP Requests)

    - Reuse HTTP sessions and connections
    - Maintains headers and connection pooling
    - Reduces latency for repeated request

Cache Invalidation Strategy

| **Cache Type** | **Invalidation Method** |
|---|---|
| Content Cache | FIFO |
| Alert Cache | Time-based expiration |
| NLP Models | Non invalidation (static) |
| Regex Cache | Reload on config change |
| DB Connections | Recycled periodically |
| Session Cache | Reset on identity rotation |

Benefits of Caching

- Fast data processing and response time
- Reduced redundant crawling and analysis
- Lower network and computational overhead
- Improved scalability of the system
- Better user experience in dashboard and alerts

# 7. Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the Dark Web Agriculture Monitor system. Unlike functional requirements, which specify what the system does, non-functional requirements describe how well the system performs its functions under various conditions.
These requirements ensure that the system is secure, reliable, scalable, and efficient when operating in real-world environments. They cover critical aspects such as system performance, data security, availability, usability, and maintainability.
By clearly defining these attributes, the system can meet expected standards for handling sensitive dark web data, maintaining anonymity, and delivering timely and accurate threat intelligence to users.

## 7.1 Security Aspects

Security is a critical requirement for the Dark Web Agriculture Monitor, as the system interacts with potentially sensitive and high-risk environments. The system is designed to ensure user anonymity, protect collected data, and secure all communications with external systems.

- Anonymity and Privacy

  The system ensures complete anonymity while accessing dark web resources

| Requirements | Implementation |
|---|---|
| IP Anonymity | All requests routed through Tor network. |

| | |
|---|---|
| Identity Rotation | Periodic IP rotation using Tor (NEWNYM signal) |
| DNS Leak Prevention | Remote DNS resolution via SOCKS5h |
| Safe Crawling | Read-only requests |

- Data Security

Sensitive data collection from dark web sources is handled carefully to prevent exposure.

| Data Type | Protection Mechanism |
|---|---|
| Credentials | Stored in redacted form |
| Database | Access controlled via authentication |
| Logs | Restricted access and periodic rotation |
| Configuration Files | Secured with limited file permissions |

- Network Security

All external communication are secured to prevent interception and misuse.

| Components | Security Measure |
|---|---|
| Tor Communication | Encrypted routing via Tor circuits |
| Email Alerts | SMTP with STARTTLS |
| Webhooks | HTTPS-based communication |
| API Access | Restricted endpoints |

- Access Control

Access to system components is controlled to prevent unauthorized page

Dashboard access is currently limited to trusted users

Future implementations includes authentication mechanism such as JWT or OAuth2

- Security Risks and Mitigation

| Risk | Mitigation |
|---|---|
| Exposure of system IP | Use of Tor network |

| | |
|---|---|
| Unauthorized dashboard access | Implement authentication |
| Data leakage | Redaction and controlled storage |
| API abuse | Rate limiting (future Enhancement |

## 7.2 Performance Aspects

The performance requirements define how efficiently the Dark Web Agriculture Monitor processes data, responds to user requests, and utilizes system resources. These requirements ensure the system operates within acceptable time limits while handling continuous dark web monitoring workloads.

### 7.2.1 Performance Objective

The system is designed to achieve the following performance goals:

- Provide fast crawling and analysis of dark web pages
- Ensure low-latency dashboard responses for users
- Minimize processing delays in NLP and alert generation
- Optimize resource utilization (CPU, memory, network)
- Maintain consistent performance under moderate load

### 7.2.2 Key Performance Metrics

| Metric | Target | Description |
|---|---|---|
| Page Crawl Time | 10 seconds | Time to fetch a dark web page via Tor |
| Content Preprocessing | < 500 ms | Cleaning and parsing HTML/text |
| Keyword Detection | < 200 ms | Pattern matching execution time |
| NLP Analysis | < 5 sec | Entity extraction and classification |
| End-to-End Processing | < 15 sec | Full pipeline per page |
| API Response Time | < 200 ms | Dashboard API latency |
| Alert Generation | < 100 ms | Time to create alert after detection |
| DB Query Time | < 50 ms | Indexed query performance |
| WebScoket Delivery | < 500 ms | Real-time alert push delay |

### 7.2.3 Throughput Capacity

The system support the following processing capacity:

| Metric | Value |
|---|---|
| Page per Hour | ~ 300 - 600 |
| Page per Day | ~ 7,000 – 14,000 |
| Alerts per Day | ~ 50 - 500 |

### 7.2.4 Resource Utilization

| Resource | Usage | Notes |
|---|---|---|
| CPU | 30-50% | Peak during NLP processing |
| Memory | 2-3 GB | NLP models consume majority |
| Network | 1-5 Mbps | Limited by Tor bandwidth |
| Disk | < 5GB | Depends on storage backend |

### 7.2.5 Performance Optimization Techniques

1. Caching
   - NLP models loaded once (reduces repeated load time)
   - Regex patterns precompiled
   - Content deduplication avoids reprocessing
2. Connection Pooling
   - Database connection reuse (QueuePool)
   - HTTP session reuse for Tor requests
3. Content Deduplication
   - Hash-based detection avoids duplicate crawling
   - Reduces workload by ~15–25%
4. Batch Processing
   - Processes multiple pages in batches
   - Improves CPU efficiency

7.2.6 Scalability Consideration

Vertical Scaling

- Increase CPU/RAM for NLP workloads
- Use GPU acceleration for large models

Horizontal Scaling

- Multiple crawler instance
- Distributed caching using Redis
- Shared database

Scalable Components

- Crawlers (Stateless, easily replicated)
- Dashboard (load-balanced)
- Alert processing (queue-based)

7.2.7 Performance Constraints

The system performance is affected by:

- Tor Network Latency
- Dark Web Site Availability
- Large NLP Models
- Single-instance limitations

## 8. References

1. Chen, H., & Wang, F. (2021). "Dark Web Monitoring for Critical Infrastructure Protection." *Journal of Cybersecurity Research*, 12(3), 45-67.
2. Kumar, A., & Singh, R. (2022). "NLP-based Threat Detection on Dark Web Platforms." *International Conference on Cyber Security*, 112-128.
3. United Nations FAO. (2022). *Digital Agriculture and Cybersecurity Guidelines*. https://www.fao.org/
4. IBM Security. (2023). *Agricultural Sector Cybersecurity Threat Intelligence Report*.